

Backend Report

Exactly what has to be built on the backend so development can be planned — the real system that replaces the MSW mocks and delivers the prize feature: a genuine on-chain settlement.

Audience	Deadline	Track	Basis
Emerson (owner)	2026-07-19	Consumer & Fan (+ Prediction Markets)	TxLINE OpenAPI 1.5.6 · txoracle IDL

Read this first. Two things are due **today**, not later: (1) the feed recorder must already be running to disk — live World Cup matches disappear with the tournament and the API's history window is too short to reconstruct them; (2) the real CPI settlement must be proven on **day 1** before anything is built on top of it. If either slips, the whole week becomes debt. Every other item can move.

1 · What the frontend already assumes

The app is built and talks to six network seams through one typed client (`src/shared/api/client.ts`), each validated by Zod (`schemas.ts`). MSW fakes them today. The real backend must return the **exact same shapes** at the same paths — this is the single swap point, nothing else in the app changes.

Frontend call	Method & path	Real backend must return
Connect wallet	POST <code>/api/wallet/connect</code>	<code>{ address, balanceSol, chain, provider }</code> — Phantom pubkey + SOL balance from RPC
Live feed	GET <code>/api/feed/:matchId</code>	<code>MatchSnapshot</code> : clock, period, score, <code>pct</code> , <code>events[]</code> , <code>markets[]</code> , live
Commit prediction	POST <code>/api/predictions</code>	<code>Prediction</code> with on-chain <code>stamp{txHash, stampedAt, seq≥1, epochDay}</code>
Poll prediction	GET <code>/api/predictions/:id</code>	same, plus <code>settlement{proofId, payoutSol, calledSecondsBefore}</code> once resolved
Profile	GET <code>/api/me?address=</code>	<code>{ accuracy, totalCalls, wonCalls, bestStreak, currentStreak, rank, balanceSol }</code>
Leaderboard	GET <code>/api/leaderboard?address=</code>	<code>{ entries[{ rank, handle, accuracy, streak, calls, you }] }</code>

Non-negotiables the schema enforces: `seq ≥ 1` (`seq=0` is rejected on-chain), `epochDay` must come from the proof timestamp, and only `provable` markets may reach settlement.

2 · The seam map — mock endpoint → real TxLINE work

Mocked endpoint	Real TxLINE endpoint(s)	Backend work
POST <code>/api/wallet/connect</code>	Solana RPC (balance) · <code>/auth/guest/start</code>	Read wallet pubkey + SOL; hold the guest JWT.
GET <code>/api/feed/:matchId</code>	<code>/scores/stream</code> + <code>/odds/stream</code> (SSE) · <code>/scores/snapshot</code> · <code>/odds/snapshot</code>	Ingest, record, and project into <code>MatchSnapshot</code> . <code>pct</code> straight from the odds <code>Pct</code> field.
POST <code>/api/predictions</code>	on-chain txoracle (stamp) — the app's commit tx	Stamp the commit; return real <code>txHash/seq/epochDay</code> .
GET <code>/api/predictions/:id</code>	<code>/scores/stat-validation-v3</code> + txoracle <code>validate_stat</code> CPI	Fetch proof, run CPI, release pot, return settlement.
GET <code>/api/me</code>	derived from recorded predictions + settlements	Aggregate accuracy/streak/rank from the call history.
GET <code>/api/leaderboard</code>	derived; ordering provable via <code>validate_stat</code>	Rank predictors; "closest call" is provable on-chain.

3 · Components to build

(a) Wallet + subscription bootstrap

Service wallet with SOL (devnet: airdrop). `subscribe(SL 12, 4 weeks)` on `txoracle` (Token-2022 ATAs; PDAs `token_treasury_v2`, `pricing_matrix`). Sign activation message, call `/api/token/activate`, store `{jwt, apiToken}`. Backs `/api/wallet/connect`.

(b) Feed ingester / recorder TODAY

SSE from `/scores/stream` + `/odds/stream`, **persisted append-only to disk** keyed by `fixtureId/seq/ts`. This is the system of record — the API's history is too short to rebuild later (trap #2). Auto-reconnect; renew JWT on 401.

(c) Match-state projector

Fold recorded score events into `MatchSnapshot`: clock, period, score, event ticker, `pct` from the clean `Pct`, and markets **discovered from the payload** — never a hardcoded catalog (trap #3). Backs `/api/feed/:matchId`.

(d) Settlement service PRIZE - DAY 1

On resolution: `GET /scores/stat-validation-v3` for the market's keys × period, then a **real CPI** into `txoracle` `validate_stat` that re-hashes the multiproof to the on-chain batch root and **releases the pot on the boolean**. No public example does this — it is the demo.

(e) Escrow / pot

Holds stakes, pays winners from the settlement boolean. **Regulatory flag:** real-money escrow needs licensing — for the hackathon ship **free-to-play / points or a sponsor pool** (cards & corners free-to-play sidesteps regulation entirely). Real money is post-hackathon, out of scope.

(f) Credential lifecycle

401 → renew JWT on the same host, retry with the same API token. 403 → assert message / wallet / signature / network / host all match one network. Never log or commit JWT / apiToken / keys.

4 • Settlement in one picture — the crux

Only **8 numbers** can ever settle on-chain: goals, yellow, red, corners — per team, per period. Everything else the feed carries (author, minute, VAR, fouls, possession) is narration for the UI and has no proof. The provable market keys are already in the app (`entities/prediction/markets.ts`, `entities/match/periods.ts`).

Base key	Stat (team 1 / 2)		Prefix	Period
1 / 2	Goals	PROVABLE	0	Total (full match)
3 / 4	Yellow cards	PROVABLE	1000	1st half
5 / 6	Red cards	PROVABLE	3000	2nd half
7 / 8	Corners	PROVABLE	4000 / 5000	Extra time
—	Foul / author / minute / VAR	UI ONLY	6000	Penalty shootout
			7000	Extra-time total

Settlement key = `prefix + baseKey`. The proof endpoint `GET /api/scores/stat-validation-v3?fixtureId=&seq=&statKeys=` (1–5 keys, both auth headers) returns a three-level Merkle bundle — individual stats → score event → fixture summary → on-chain batch root — as `ScoresStatValidationV3 { ts, summary, eventStatRoot, statsToProve[], multiproof, subTreeProof, mainTreeProof }`. `seq` comes from the event and `epochDay` from the proof `ts` — never `Date.now()`.

The exact CPI instruction, args, and accounts must be read from the network's `txoracle.json` IDL — the IDL is the authority. Official examples only ever run it as a `.view()` simulation; performing a real CPI that releases value on the returned boolean is the one thing no public example demonstrates, and the strongest possible demo.

5 • Order of attack — day by day

When	Deliverable
TODAY	Service wallet + SOL → <code>subscribe</code> SL 12 (not 1) → activate token → feed recorder running to disk . Nothing else starts until this runs.
DAY 1	De-risk the CPI first: a minimal Anchor program doing a real CPI into <code>validate_stat</code> and reading the boolean. Works → project stands. Fails → better to know now than on day 6.
Days 2–3	Pot contract: deposit → predict → settlement via CPI → payout. One match, one pot, no frills.
Days 4–5	Wire the real backend behind the six seams (swap MSW). Projector + prediction/settlement endpoints. This is where UX is judged.
Day 6	Run against a real live match, real people, real (points) pot. Find what breaks.
Day 7	Demo video — eliminatory requirement, always runs late. Reserve the whole day. Submit to all listings.

6 · Env, libraries, risks

Env / secrets

`NETWORK` · `SOLANA_RPC_URL` · `TXLINE_API_ORIGIN` ·
`TXORACLE_PROGRAM_ID` · `TXL_TOKEN_MINT` ·
`SERVICE_WALLET_SECRET` (never commit) · `TXLINE_JWT`
 (refresh on 401) · `TXLINE_API_TOKEN` .

Libraries

`@coral-xyz/anchor` , `@solana/web3.js` , `@solana/spl-token` (`TOKEN_2022`), `tweetnacl` , `axios` , an SSE client.
Node 20+. IDL `txoracle.json` + `Txoracle` type must match the chosen network.

Risk	Mitigation
CPI has no public example	Prove it day 1. Fallback: <code>.view()</code> simulation + signed settlement — works, less impressive.
Mainnet vs devnet mixing → silent 403	One network per credential set. Real-time World Cup only on mainnet SL 12 ; devnet for cheap CPI proving.
Regulatory (real-money escrow)	Ship free-to-play / points / sponsor pool for the hackathon. Real money needs licensing — out of scope.
History window too short	Recorder must run before matches, or the data is gone. That's why it's today.

The only way to lose by carelessness. The rules require a working build with video — mockups and pitch decks don't count, and live matches end with the World Cup. If the feed recorder isn't running now and the CPI isn't proven on day 1, the week turns into debt. Those two are today.

Called It — backend planning report. Sources: TxLINE OpenAPI 1.5.6, quickstart, World Cup free-tier guide, txoracle IDL, project strategy doc. Programs — mainnet `9ExbZjAapQww1vfcisDmrngPinHTEfpjYRWMunJgcKaA` , devnet `6pW64gN1s2uqjHkn1unFeEjAwJkPGHopp6vS715wyp2J` . See docs/txline-integration.md and docs/BACKEND.md.